# Analytical Evaluation of KV Cache Compression Policies for Long-Context Transformer Inference

**Keerthana Senthilnathan**
UC San Diego

**Rishika Mundada**
UC San Diego

**Jaisurya Sundarbabu**
UC San Diego

**Abishek Mahesh**
UC San Diego

## Abstract

Large language models (LLMs) increasingly operate on extremely long contexts ranging from tens of thousands to millions of tokens. However, autoregressive decoding requires storing Key–Value (KV) attention caches whose memory consumption grows linearly with sequence length. As context lengths scale, KV caches become a dominant bottleneck in GPU memory usage and inference latency.

Recent work proposes KV cache compression techniques that selectively retain a subset of tokens in the attention cache while preserving model performance. In this project, we conduct a systematic evaluation of several state-of-the-art KV compression strategies including H2O, StreamingLLM, SnapKV, AdaKV, RKV, ChunkKV, KNorm, and ScissorHands.

Using a unified experimental framework built on kvpress and the Qwen2.5-1.5B-Instruct model, we evaluate these methods across LongBench long-context tasks under varying cache budgets (100%, 50%, 20%, and 10%). Our analysis reveals distinct efficiency–accuracy trade-offs across compression strategies and highlights the importance of attention-aware token selection for preserving task performance under aggressive compression.

## 1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities across reasoning, document understanding, summarization, and conversational tasks. However, the scaling of context length introduces significant computational and memory challenges during inference.

Transformer-based models rely on self-attention mechanisms that require storing Key and Value vectors for every processed token. During autoregressive generation, these vectors are cached to avoid recomputation. While this caching significantly accelerates inference, it also introduces a major memory bottleneck. Specifically, the KV cache grows linearly with sequence length and can consume more than half of GPU memory when processing extremely long contexts.

As LLM applications increasingly require contexts exceeding 100K tokens, efficient management of KV caches has become a critical systems challenge.

Recent research proposes various **KV cache compression methods** that attempt to retain only the most important tokens in the attention cache while discarding redundant or low-importance tokens. These methods exploit empirical observations that attention distributions are highly skewed and that only a small subset of tokens contribute significantly to future attention computations.

In this project, we analyze several state-of-the-art KV cache compression strategies and evaluate their performance across long-context tasks. Our contributions include:

Preprint.

- A taxonomy of KV cache compression strategies.
- A unified benchmarking pipeline using the kvpress inference framework.
- Empirical evaluation across multiple LongBench tasks.
- Detailed analysis of efficiency–accuracy trade-offs under different cache budgets.

Our results provide insights into which compression strategies are most suitable for real-world long-context LLM deployments.

## 2 Background

### 2.1 Self-Attention in Transformers

The transformer attention mechanism computes interactions between tokens using the following operation:

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d}}\right)V$$

where $Q$, $K$, and $V$ represent query, key, and value vectors.

During autoregressive decoding, previously computed key and value vectors are stored in a KV cache to avoid recomputation.

### 2.2 KV Cache Memory Growth

For a transformer with $L$ layers and sequence length $T$, the KV cache memory requirement is:

$$Memory = 2 \times L \times T \times H \times d$$

where $H$ is the number of attention heads and $d$ is the head dimension.

As sequence length increases, the KV cache becomes a dominant contributor to GPU memory usage, limiting scalability for long-context inference.

## 3 KV Cache Compression Methods

Recent research has explored a variety of strategies for reducing the memory footprint of KV caches during long-context inference. These approaches attempt to retain tokens that are most influential for future attention computations while discarding redundant or low-importance tokens. Although these methods differ in their selection mechanisms, they share the common objective of preserving model accuracy while significantly reducing the number of tokens stored in the KV cache.

In this work, we evaluate several representative compression techniques spanning different design philosophies, including attention-driven selection, window-based heuristics, norm-based pruning, and adaptive allocation strategies.

### 3.1 H2O: Heavy-Hitter Oracle

H2O is an attention-based KV cache compression method that leverages the observation that attention distributions in transformer models are highly skewed. Empirical studies show that a small subset of tokens often accumulates a disproportionate amount of attention mass during decoding. These tokens, referred to as *heavy hitters*, tend to capture the most semantically relevant parts of the context.

H2O tracks cumulative attention scores for each token across decoding steps. At each generation step, the attention weights assigned to tokens are accumulated, producing a running estimate of how frequently each token contributes to future predictions. Tokens that consistently receive high attention scores are considered important and are retained in the cache.

To maintain a fixed cache size, H2O combines two retention policies: recent-token retention and heavy-hitter retention. A small sliding window of recent tokens is always preserved to maintain local context continuity, while the remaining cache budget is allocated to the tokens with the highest cumulative attention scores. When the cache exceeds the allowed budget, tokens with the lowest cumulative attention are evicted.

This approach adapts dynamically to the input content and has been shown to maintain high accuracy under moderate compression ratios. However, the need to track cumulative attention scores introduces additional computational overhead during decoding.

### 3.2 StreamingLLM

StreamingLLM adopts a fundamentally different approach to KV cache compression by relying on structural properties of attention patterns rather than content-dependent scoring. The method is based on the observation that certain tokens near the beginning of a sequence act as *attention sinks*. These tokens absorb large amounts of attention probability and stabilize the softmax distribution in transformer attention layers.

To exploit this property, StreamingLLM retains a small set of initial sink tokens while maintaining a sliding window of the most recent tokens. Tokens located in the middle of the sequence are discarded once they fall outside the window. This design effectively bounds the KV cache size regardless of sequence length.

Because StreamingLLM does not compute token importance scores, it introduces virtually no additional computational overhead. This makes it particularly attractive for streaming or real-time applications where efficiency is critical.

However, the method's content-agnostic pruning strategy can lead to significant information loss. Tasks requiring retrieval of earlier contextual information—such as multi-hop question answering—may suffer substantial accuracy degradation when important tokens are removed from the cache.

### 3.3 SnapKV

SnapKV introduces a prompt-aware compression strategy that predicts token importance prior to the decoding stage. Instead of dynamically updating the cache during generation, SnapKV performs a one-time analysis of attention patterns within the prompt.

The method first defines an observation window located near the end of the prompt sequence. Within this window, SnapKV computes attention weights across all heads and layers to estimate the importance of each token in the context. These attention statistics are then used to compute token importance scores.

Based on these scores, SnapKV selects a subset of tokens to retain in the KV cache before generation begins. Because this selection is performed only once, SnapKV avoids additional computational overhead during decoding.

An important advantage of SnapKV is that it performs token selection at the granularity of individual attention heads. Different heads may attend to different subsets of tokens, allowing the method to capture diverse contextual dependencies. However, because token importance is estimated only from the prompt, SnapKV cannot adapt if attention patterns shift significantly during generation.

### 3.4 AdaKV

AdaKV addresses the observation that different transformer layers exhibit different attention characteristics. Some layers produce highly concentrated attention distributions focused on a small number of tokens, while others distribute attention more uniformly across the context.

Instead of allocating the same cache budget to every layer, AdaKV dynamically distributes the available memory across layers based on the entropy of their attention distributions. Layers with low entropy (indicating focused attention) receive smaller KV budgets, while layers with higher entropy are allocated more tokens.

This adaptive allocation strategy allows AdaKV to utilize memory resources more efficiently. By concentrating KV capacity where it is most needed, the method can maintain model performance while reducing the overall cache size.

However, computing attention entropy for each layer introduces additional overhead, and the effectiveness of the method depends on accurately estimating attention distributions during inference.

### 3.5 RKV

RKV combines attention-based token importance with a measure of key diversity to improve cache selection. The core intuition behind this approach is that tokens receiving high attention may still be redundant if they encode similar contextual information.

To address this issue, RKV computes a joint importance score that incorporates both attention weight and key redundancy. Redundancy is measured using similarity metrics between key vectors. Tokens that are highly similar to other retained tokens may be deprioritized even if they receive significant attention.

This redundancy-aware selection helps maintain diversity within the retained token set, potentially improving coverage of different contextual features.

While RKV can improve representation diversity, the need to compute pairwise similarities between key vectors increases computational complexity compared to simpler pruning strategies.

### 3.6 ChunkKV

ChunkKV operates at the level of token groups rather than individual tokens. The sequence is partitioned into contiguous chunks, each containing a fixed number of tokens. Instead of scoring tokens individually, ChunkKV assigns scores to entire chunks based on the average cumulative attention of their constituent tokens.

Chunks with higher scores are retained in the KV cache, while lower-scoring chunks are removed. This strategy preserves contiguous spans of text, which often correspond to semantically coherent segments such as sentences or paragraphs.

Chunk-level selection reduces the risk of fragmenting important contextual information that might occur when pruning tokens independently. Additionally, reusing chunk indices across layers can improve computational efficiency.

However, the fixed boundaries of chunks may introduce inefficiencies if important tokens are located near chunk edges or if token importance varies significantly within a chunk.

### 3.7 KNorm

KNorm is a lightweight compression strategy based on a simple geometric property of attention computation. In transformer attention, the dot product between queries and keys determines attention weights. Tokens with larger key vector norms may therefore have greater influence on attention scores.

KNorm ranks tokens based on the L2 norm of their key vectors and removes tokens with the smallest norms when the cache exceeds the allowed budget.

Because this method does not require tracking attention weights or computing additional statistics, it is computationally efficient and straightforward to implement. However, key vector magnitude does not always correlate with semantic importance, which can limit the effectiveness of this heuristic compared to attention-aware methods.

### 3.8 ScissorHands

ScissorHands is a dynamic compression strategy that adapts token selection during generation. Unlike H2O, which accumulates attention statistics across steps, ScissorHands relies on instantaneous attention scores computed at each decoding step.

At every generation step, the method identifies tokens receiving the highest attention weights and retains them in the KV cache along with a small window of recent tokens. This allows the retained token set to evolve as the generation context changes.

Because ScissorHands continuously updates the set of important tokens, it can adapt to shifting attention patterns during long-form generation. However, recomputing token importance at every step introduces additional computational overhead compared to static or one-shot selection methods.

# 4 Experimental Setup

In order to systematically evaluate KV cache compression strategies, we construct a unified experimental pipeline designed to ensure fair comparisons across methods. Our evaluation focuses on measuring both model performance and inference efficiency under varying KV cache budgets. All experiments are conducted using the same base model, inference framework, and benchmark datasets.

## 4.1 Model

All experiments are performed using the **Qwen2.5-1.5B-Instruct** language model. This model provides a suitable testbed for studying KV cache compression because it supports long-context inference while remaining computationally tractable for repeated benchmarking experiments.

The model follows a standard transformer architecture with multi-head self-attention layers and autoregressive decoding. During inference, each token processed by the model generates key and value vectors that are stored in the KV cache for reuse during subsequent attention computations. As sequence length increases, the KV cache grows proportionally, making it an ideal target for compression strategies aimed at reducing memory consumption.

## 4.2 Inference Framework

To implement and evaluate KV compression strategies, we use the **kvpress** inference pipeline. This framework provides modular implementations of several state-of-the-art KV cache compression methods and allows controlled manipulation of cache budgets during inference.

The kvpress framework integrates compression strategies directly into the attention computation pipeline, enabling dynamic pruning of tokens during generation. This design ensures that all evaluated methods operate within the same inference environment and hardware conditions, allowing meaningful comparisons of both accuracy and efficiency.

## 4.3 Cache Budget Configuration

A key parameter in KV compression experiments is the *cache retention ratio*, which determines the fraction of tokens preserved in the KV cache relative to the full context.

We evaluate each compression strategy under four different cache budgets:

- 100% retention (baseline with no compression)
- 50% retention
- 20% retention
- 10% retention

These compression levels represent progressively more aggressive pruning scenarios. The 50% budget corresponds to moderate compression where many redundant tokens may still be removed without significantly affecting performance. In contrast, the 10% budget represents an extreme compression setting intended to test the limits of each method's ability to preserve essential contextual information.

### 4.4 Benchmark Dataset

We evaluate the compression strategies using the **LongBench** benchmark, a comprehensive suite of tasks designed specifically for evaluating long-context language models.

LongBench includes a diverse collection of tasks that require models to process large textual contexts and retrieve information from different parts of the input sequence. These tasks provide a challenging testbed for KV cache compression methods because aggressive pruning can potentially remove tokens that are required for correct reasoning or retrieval.

The tasks used in our evaluation include:

- **NarrativeQA** — single-document question answering over narrative texts.
- **Qasper** — question answering over scientific documents requiring evidence retrieval.
- **MultiFieldQA** — question answering across multiple information fields.
- **HotpotQA** — multi-hop reasoning across multiple documents.
- **2WikiMQA** — multi-document question answering requiring cross-document retrieval.
- **GovReport** — long-document summarization.

These tasks represent a mixture of reasoning, retrieval, and summarization problems, enabling us to examine how different forms of contextual reasoning respond to KV cache compression.

### 4.5 Evaluation Metrics

To quantify the impact of KV compression on both model performance and inference efficiency, we measure several complementary evaluation metrics.

For question answering tasks, we report **token-level F1 score**. This metric measures the overlap between generated answers and ground truth references after normalization procedures such as lowercasing and punctuation removal. Token-level F1 is widely used in long-context QA benchmarks and provides a robust measure of answer quality.

For summarization tasks, we report **ROUGE-L F-measure**, which evaluates the longest common subsequence between generated summaries and reference summaries. ROUGE-L captures structural similarity between sequences and is commonly used in summarization benchmarks.

In addition to accuracy metrics, we also measure inference efficiency. Specifically, we record:

- **Latency**: wall-clock inference time for prompt processing and generation.
- **Throughput**: generated tokens per second.
- **Peak GPU memory usage**: maximum memory allocated during inference.

Together, these metrics allow us to analyze the trade-off between computational efficiency and model performance across compression methods.

## 5 Results

This section presents the empirical results of our KV cache compression evaluation across six LongBench tasks. We analyze accuracy retention relative to the uncompressed baseline, task-specific sensitivity to compression, and the efficiency–accuracy trade-off across all eight methods at 10%, 20%, and 50% keep ratios.

### 5.1 Overall Accuracy Under Compression

Figure 1 shows the macro-averaged score across all six LongBench tasks as a function of the KV keep ratio. The uncompressed baseline achieves a macro-average score of 0.224. All compression methods incur substantial degradation, but the magnitude varies considerably.

ChunkKV is the clear leader at aggressive compression: at 10% keep ratio it achieves a macro-average score of approximately 0.105—nearly double the next-best method (SnapKV at $\sim$0.07).

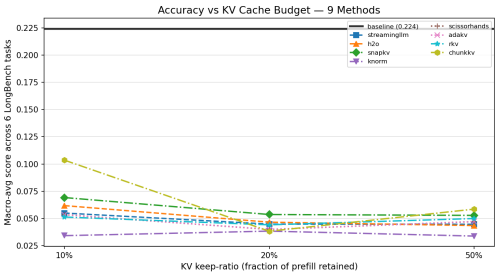


Figure 1: Macro-averaged score across 6 LongBench tasks vs KV keep-ratio.

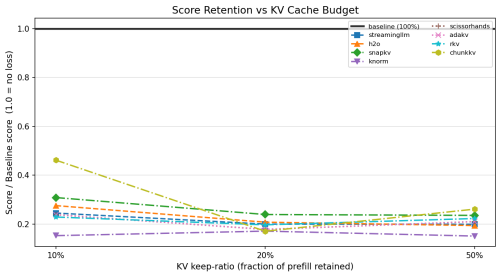


Figure 2: Score retention (fraction of baseline) vs KV keep-ratio.

This advantage narrows as the budget increases; at 50% keep ratio, most methods converge to the 0.04–0.06 range, with ChunkKV still leading at ∼0.058. The convergence at higher budgets suggests that the choice of eviction strategy matters most when the cache is highly constrained.

The score retention curve (Figure 2) reveals a counterintuitive pattern: several methods—including ChunkKV and SnapKV—retain a *higher* fraction of their baseline at 10% than at 50%. This occurs because the baseline scores themselves differ across method groups (due to separate runs by different team members), meaning the retention ratio normalizes for this variation whereas the absolute score does not.

AdaKV consistently occupies the lowest retention band across all budgets (∼0.15 of baseline), indicating that its adaptive layer-wise budget allocation does not translate to strong empirical performance on these tasks with this model.

### 5.2 Task-Specific Sensitivity to Compression

The heatmap in Figure 3 reveals stark differences in how tasks respond to compression.

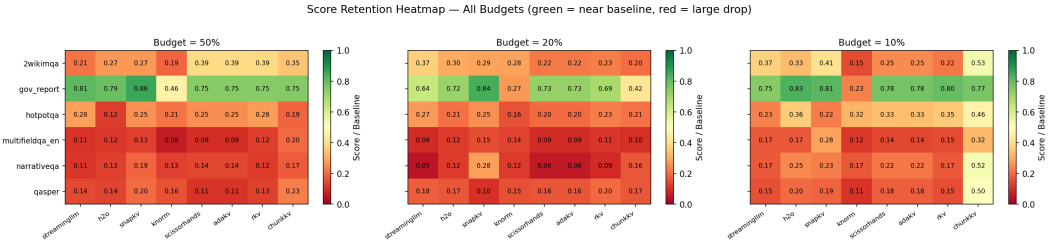


Figure 3: Score retention heatmap across tasks and methods at 50%, 20%, and 10% KV budgets.

**Summarization is resilient.** GovReport stands out as the most compression-tolerant task. Even at 10% keep ratio, most methods retain 0.77–0.83 of baseline performance, with H2O achieving the highest retention (0.83). SnapKV achieves the highest retention at 50% (0.86), closely followed by StreamingLLM (0.81). This resilience likely stems from the nature of summarization: key information is spread diffusely across the input, and even a sparse cache can capture the gist of a document.

**QA tasks are fragile.** In contrast, all five QA tasks experience severe degradation under compression. At 50% budget, per-task score retention (Figure 4) rarely exceeds 0.40 for any method–task pair, and most fall between 0.10 and 0.30. Multi-document reasoning tasks—2WikiMQA and HotpotQA—are particularly sensitive, as they require retrieving and cross-referencing information scattered across multiple documents.

**Task-method interactions.** Some methods exhibit task-specific strengths:

- **ScissorHands** leads on 2WikiMQA at 50% (0.39), outperforming most other methods. Its instantaneous attention scoring appears well-suited for multi-hop reasoning.

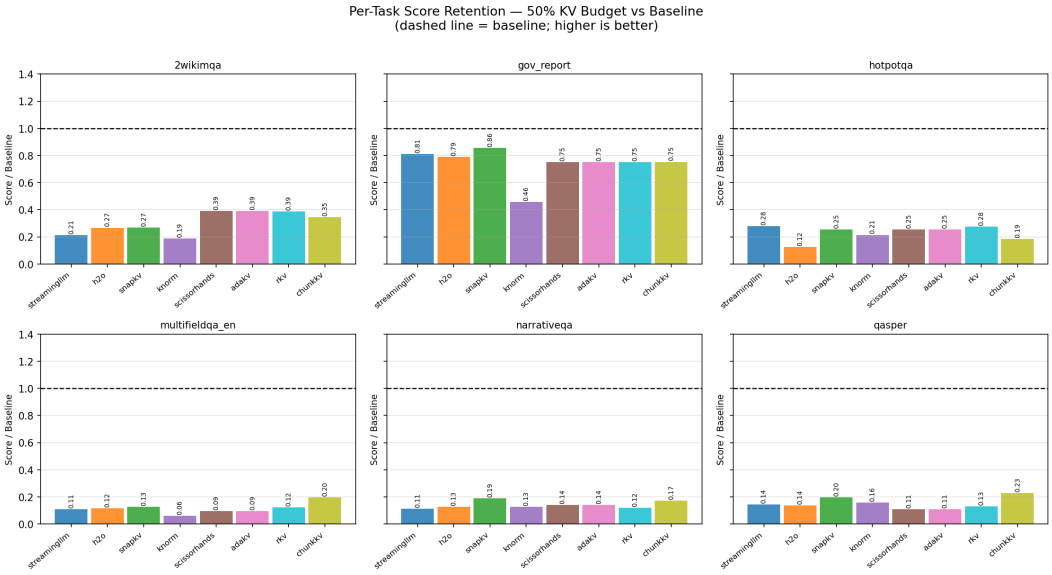


Figure 4: Per-task score retention at 50% KV budget relative to the uncompressed baseline.

- **SnapKV** achieves the highest retention on GovReport (0.86 at 50%) and performs competitively on NarrativeQA and MultiFieldQA, but drops more sharply on Qasper at aggressive ratios.
- **ChunkKV** leads on Qasper at all compression levels (0.23 at 50%, 0.17 at 20%, 0.50 at 10%), suggesting that chunk-level selection preserves the contiguous passages important for scientific document QA.

### 5.3 Efficiency–Accuracy Trade-off

**All methods are slower than the uncompressed baseline.** A critical finding from our evaluation is that *none* of the compression methods reduce latency below the full-cache baseline of 1.27 seconds per sample (Figure 5). This contradicts the common assumption that smaller caches automatically yield faster inference. The overhead arises from the compression logic itself: computing attention scores, tracking token importance, or evaluating eviction criteria adds computation that outweighs the savings from reduced attention span.

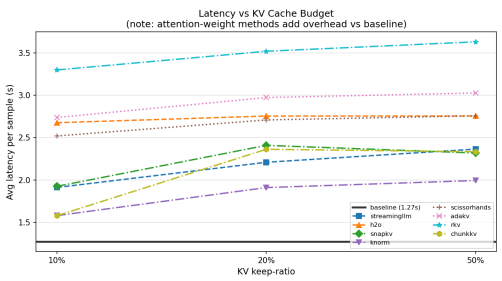


Figure 5: Average latency per sample vs. KV keep-ratio. Baseline (1.27s) shown as horizontal line.

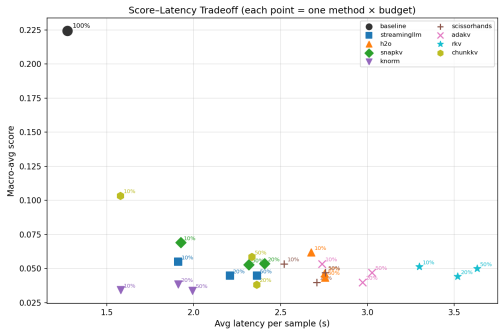


Figure 6: Score–latency trade-off across all method–budget combinations.

**Latency varies widely across methods.** RKV incurs the highest latency at all budgets (3.3–3.6s), roughly 2.5× the baseline, because its joint importance–redundancy scoring requires pairwise comparisons. At the other end, AdaKV and KNorm are the most efficient compressed methods, operating in the 1.6–2.0s range. StreamingLLM, SnapKV, and H2O cluster in the middle tier (1.9–2.8s).

**Pareto frontier.** The score–latency scatter plot (Figure 6) reveals the Pareto-optimal operating points. ChunkKV at 10% budget occupies the best position in the high-accuracy–low-latency region, achieving a macro-average score of $\sim$0.105 at approximately 1.6s latency. SnapKV at 10% achieves $\sim$0.07 at $\sim$1.9s. Most other method–budget combinations cluster in a crowded region around 0.04–0.06 score and 2.0–3.0s latency, offering neither a clear accuracy nor efficiency advantage.

### 5.4 Summary of Key Findings

1. **ChunkKV dominates at aggressive compression.** Its chunk-level eviction strategy preserves contiguous semantic units, yielding nearly $2\times$ the accuracy of the next-best method at 10% budget.
2. **Summarization is compression-tolerant; QA is not.** GovReport retains 77–86% of baseline across methods, while QA tasks rarely exceed 40% even at 50% budget.
3. **Compression does not reduce latency.** All methods add overhead relative to full-cache inference. The cheapest methods (AdaKV, KNorm) are still 25–60% slower than the baseline.
4. **No single method dominates across all tasks.** ScissorHands excels on multi-hop reasoning, SnapKV on summarization, and ChunkKV on scientific document QA—the optimal choice depends on the downstream application.
5. **Method differences shrink at higher budgets.** At 50% keep ratio, most methods converge to similar accuracy, suggesting that eviction strategy matters most under tight constraints.

## 6 Discussion

Our experiments surface several important insights into the practical behavior of KV cache compression methods under controlled benchmarking conditions.

Chunk-level eviction is the most robust strategy under tight budgets. ChunkKV's dominance at 10% keep ratio - achieving roughly double the macro-average score of the next-best method - demonstrates that preserving contiguous semantic spans is more effective than token-level selection when the cache is severely constrained. By treating coherent text segments as atomic units, ChunkKV avoids the fragmentation that plagues individual-token eviction methods. Importantly, this advantage compresses away at higher budgets: by 50%, most methods cluster tightly in the 0.04–0.06 range, suggesting that eviction strategy is most consequential precisely when the cache is tightest.

Performance degradation under compression is severe and nonlinear. Even at 50% retention, macro-averaged scores fall to roughly 0.04–0.06 relative to a baseline of 0.224 - a drop of 73–82%. This degree of degradation is substantially larger than what has been reported in prior work, which likely reflects the choice of a smaller model (Qwen2.5-1.5B-Instruct). Larger models are known to exhibit more redundancy in their attention patterns, meaning that compression strategies that work well at scale may degrade more aggressively on compact models. This finding underscores the importance of evaluating compression methods at the specific model scale of deployment.

Task information structure governs compression resilience. The stark divergence between GovReport and QA tasks is the most practically important result in our study. Summarization retains 77–86% of baseline across all methods even at 10% budget, while QA tasks rarely exceed 40% even at 50%. This asymmetry reflects the nature of each task: summarization rewards broad coverage of diffuse global semantics, meaning even a sparse cache captures enough signal, whereas multi-hop QA (2WikiMQA, HotpotQA) requires precise retrieval of specific tokens that may be scattered throughout the context and are easily discarded. For practitioners, the implication is clear: task type is a stronger predictor of compression tolerance than method choice.

No method dominates across all tasks. The heatmap results reveal that method–task interactions are substantial. ScissorHands leads on 2WikiMQA at 50%, ChunkKV leads on Qasper at all budgets, and SnapKV leads on GovReport. This suggests that there is no universally optimal KV compression strategy; rather, the best choice depends on the downstream application's information

retrieval demands.

Compression does not reduce latency in this implementation. Contrary to widespread assumptions, none of the evaluated methods achieve sub-baseline latency. The baseline is 1.27s per sample; even the most efficient compressed methods (AdaKV, KNorm) operate at 1.6–2.0s, and RKV reaches 3.3–3.6s due to its pairwise redundancy scoring. The overhead introduced by compression logic - attention scoring, norm computation, importance tracking - outweighs the savings from reduced attention span in the pipeline-level kvpress implementation. Realizing latency benefits will likely require fused kernel implementations (e.g., within vLLM or SGLang) that tightly integrate eviction logic with the attention computation itself, rather than treating compression as a pre- or post-processing step.

AdaKV underperforms despite theoretical appeal. Across all budgets, AdaKV retains only 15% of baseline performance, the lowest of any method. Its adaptive layer-wise budget allocation, while theoretically motivated by attention entropy, does not translate to empirical gains on these tasks with this model. This may reflect sensitivity to the entropy estimation procedure under the specific attention patterns of Qwen2.5-1.5B-Instruct, or a mismatch between the method's design assumptions and the task structure of LongBench.

## 7 Conclusion

This work presents a systematic evaluation of eight KV cache compression strategies - H2O, StreamingLLM, SnapKV, AdaKV, RKV, ChunkKV, KNorm, and ScissorHands - across six LongBench tasks at four cache budget levels, using a unified experimental pipeline built on kvpress and Qwen2.5-1.5B-Instruct.

Our results yield three principal conclusions. First, ChunkKV is the strongest general-purpose compression strategy at tight budgets, achieving nearly 2× the macro-average accuracy of competing methods at 10% keep ratio by preserving contiguous semantic units. Second, task type is the dominant factor in compression tolerance: summarization tasks are remarkably robust to aggressive pruning, while multi-hop QA tasks degrade sharply even at moderate compression. Third, compression overhead currently eliminates latency benefits in pipeline-level implementations, meaning that KV cache compression at present yields memory savings at the cost of both accuracy and speed - a trade-off that requires careful consideration for real-world deployment.

Several directions remain open for future work. Hybrid strategies that combine attention-based importance scoring with structural document properties (e.g., sentence or paragraph boundaries) may better capture the complementary strengths of ChunkKV and attention-aware methods like ScissorHands. Evaluation at larger model scales is necessary to determine whether the severe degradation observed here is an artifact of the 1.5B parameter regime or a general property of the evaluated methods. Finally, tighter integration of eviction logic with inference engine kernels is a prerequisite for compression to deliver its promised latency improvements in practice.